

Phase 1 Implementation Guide

Foundation & Architecture Setup

Version: 1.0

Date: February 3, 2026

Estimated Time: 6-8 hours

Status: Ready for Implementation

Overview

This guide provides step-by-step instructions for setting up the Christmas Tracker iOS project foundation. By the end of Phase 1, you will have:

- ✓ Xcode project configured with iOS 26 + Swift 6
 - ✓ xcconfig files for production and debug configurations
 - ✓ Complete folder structure
 - ✓ Design System (colors, typography, spacing) in Asset Catalog
 - ✓ Core protocol definitions (APIClient, Repositories, LogDestination)
 - ✓ Mock data factory for previews and tests
 - ✓ First unit tests passing
 - ✓ SwiftUI preview working with mock data
-

Prerequisites

Required Software

- **macOS Sequoia 15.0+** (or compatible version)
- **Xcode 26.0.1 or later**
- **Command Line Tools** installed
- **Git** configured

Verification

```
# Verify Xcode version
xcodebuild -version
# Expected: Xcode 26.0.1

# Verify Swift version
swift --version
# Expected: Swift version 6.0

# Verify Git
git --version
```

Step 1: Create Xcode Project

1.1 Create New Project

1. Open Xcode 26.0.1
2. Click "Create New Project"
3. Select **iOS → App**

4. Configure project:

- **Product Name:** `ChristmasTracker`
- **Team:** Select your team
- **Organization Identifier:** `com.christmastracker`
- **Bundle Identifier:** `com.christmastracker.ios`
- **Interface:** SwiftUI
- **Language:** Swift
- **Include Tests:** ☒ (checked)

5. Choose location and create project

1.2 Verify Project Settings

Navigate to **Project Settings** → **ChristmasTracker Target** → **General**:

- **Deployment Info:**

- iOS Deployment Target: `26.0`
- Supported Destinations: iPhone, iPad
- Supported Orientations: Portrait, Landscape (iPad only)

- **Identity:**

- Display Name: `Christmas Tracker`
- Bundle Identifier: `com.christmastracker.ios`
- Version: `1.0.0`
- Build: `1`

1.3 Configure Build Settings

Navigate to **Build Settings**:

- Search for "Swift Language Version"
 - Set to: **Swift 6**
- Search for "Swift Compiler - Custom Flags"
 - Add to "Other Swift Flags": `-strict-concurrency=complete`

Step 2: Create Configuration Files

2.1 Create Config Directory

```
# In project root
mkdir Config
cd Config
```

2.2 Create Config.xcconfig (Production)

Create `Config/Config.xcconfig`:

```
// Christmas Tracker – Production Configuration
// iOS 26.0 Minimum, Swift 6

// Platform
IOS_DEPLOYMENT_TARGET = 26.0
SWIFT_VERSION = 6.0
```

```
// Bundle
PRODUCT_BUNDLE_IDENTIFIER = com.christmastracker.ios
PRODUCT_NAME = $(TARGET_NAME)

// API Configuration
API_BASE_URL = https://api.christmastracker.com

// Custom App Configuration
CACHE_DURATION = 900 // 15 minutes
SESSION_TIMEOUT = 7200 // 2 hours
BACKGROUND_REFRESH = 900 // 15 minutes
LOG_FLUSH_INTERVAL = 60 // 60 seconds

// Build Settings
ENABLE_BITCODE = NO
ENABLE_TESTABILITY = NO
```

2.3 Create Config-QA.xcconfig (QA/UAT)

Create `Config/Config-QA.xcconfig`:

```
// Christmas Tracker - QA/UAT Configuration
// Includes production settings with QA overrides

#include "Config.xcconfig"

// QA Overrides
PRODUCT_BUNDLE_IDENTIFIER = com.christmastracker.ios.qa
API_BASE_URL = https://api-qa.christmastracker.com

// Shorter timeouts for QA testing
CACHE_DURATION = 60 // 1 minute
SESSION_TIMEOUT = 1800 // 30 minutes
BACKGROUND_REFRESH = 300 // 5 minutes
LOG_FLUSH_INTERVAL = 30 // 30 seconds

// Enable testability
ENABLE_TESTABILITY = YES
```

2.4 Create Config-Debug.xcconfig (Development)

Create `Config/Config-Debug.xcconfig`:

```
// Christmas Tracker - Debug Configuration
// Includes production settings with debug overrides

#include "Config.xcconfig"

// Debug Overrides
PRODUCT_BUNDLE_IDENTIFIER = com.christmastracker.ios.dev
API_BASE_URL = https://api-dev.christmastracker.com

// Debug Build Settings
ENABLE_TESTABILITY = YES
GCC_OPTIMIZATION_LEVEL = 0
SWIFT_OPTIMIZATION_LEVEL = -Onone
```

```
// Custom App Configuration (Debug Values)
CACHE_DURATION = 30           // 30 seconds (faster testing)
SESSION_TIMEOUT = 300         // 5 minutes
BACKGROUND_REFRESH = 60       // 1 minute
LOG_FLUSH_INTERVAL = 15       // 15 seconds
```

2.5 Link xcconfig to Schemes

1. Open **Project Settings** (click project name in navigator)
2. Select **Project (not target)** "ChristmasTracker"
3. Go to **Info** tab
4. Under **Configurations**, expand each configuration:
 - **Debug:**
 - Set all targets to: **Config/Config-Debug**
 - **QA/UAT (if you create this configuration):**
 - Set all targets to: **Config/Config-QA**
 - **Release:**
 - Set all targets to: **Config/Config**

To create QA configuration:

1. Click **+** button under Configurations
2. Duplicate **Release**
3. Rename to **QA**
4. Assign **Config-QA.xcconfig**

2.6 Add Configuration Values to Info.plist

1. Open **Info.plist**
2. Add these keys (right-click → Add Row):

```
<key>API_BASE_URL</key>
<string>$(API_BASE_URL)</string>
<key>CACHE_DURATION</key>
<string>$(CACHE_DURATION)</string>
<key>SESSION_TIMEOUT</key>
<string>$(SESSION_TIMEOUT)</string>
<key>BACKGROUND_REFRESH</key>
<string>$(BACKGROUND_REFRESH)</string>
<key>LOG_FLUSH_INTERVAL</key>
<string>$(LOG_FLUSH_INTERVAL)</string>
```

Step 3: Create Folder Structure

3.1 Delete Default Files

1. Delete **ContentView.swift** (we'll create organized views)
2. Keep **ChristmasTrackerApp.swift**

3.2 Create Folder Groups in Xcode

Right-click on **ChristmasTracker** folder → New Group:

Create these groups (use "New Group" not "New Group from Folder"):

```
ChristmasTracker/
├── App/
├── Core/
│   ├── Configuration/
│   ├── Networking/
│   ├── Security/
│   └── Logging/
├── Features/
│   ├── Authentication/
│   │   ├── Views/
│   │   ├── Services/
│   │   ├── Repositories/
│   │   └── Models/
│   ├── Dashboard/
│   ├── Lists/
│   ├── Items/
│   ├── Statistics/
│   ├── Profile/
│   └── Notifications/
├── DesignSystem/
│   ├── Colors/
│   ├── Typography/
│   ├── Spacing/
│   ├── Components/
│   └── Icons/
├── Navigation/
├── Utilities/
│   ├── Extensions/
│   ├── Helpers/
│   └── Validation/
└── Preview Content/
```

Important: These are Xcode groups (virtual), not filesystem folders. Files will be organized in groups but all live in the same directory initially.

Step 4: Set Up Asset Catalog

4.1 Configure Assets.xcassets

1. Open **Assets.xcassets**
2. Create folder structure:

```
Assets.xcassets/
├── Colors/
│   ├── Primary/
│   ├── Accent/
│   ├── Highlight/
│   ├── Background/
│   ├── Surface/
│   ├── Text/
│   ├── Border/
│   ├── Status/
│   └── Budget/
├── AppIcon.appiconset/
└── (existing files)
```

4.2 Add Primary Color

1. Right-click **Colors/Primary/** → New Color Set

2. Name it **primary**

3. Select the color set

4. In Attributes Inspector (right panel):

- **Appearances:** Any, Dark
- **Devices:** Universal

5. Set Light Appearance:

- Color: RGB Sliders
- Red: **31** (0.122)
- Green: **77** (0.302)
- Blue: **58** (0.227)
- Alpha: **255** (1.0)

6. Set Dark Appearance:

- Select "Dark" in Attributes
- Color: RGB Sliders
- Red: **79** (0.310)
- Green: **165** (0.647)
- Blue: **138** (0.541)
- Alpha: **255** (1.0)

4.3 Add All Brand Colors

Repeat for each color using values from Design System Specification:

Accent (Cranberry):

- Light: **#B23A48** (RGB: 178, 58, 72)
- Dark: **#E06A75** (RGB: 224, 106, 117)

Highlight (Warm Gold):

- Light: **#D6B25E** (RGB: 214, 178, 94)
- Dark: **#E4C76A** (RGB: 228, 199, 106)

Background Primary:

- Light: **#FAF9F7** (RGB: 250, 249, 247)
- Dark: **#0F1A17** (RGB: 15, 26, 23)

Background Secondary:

- Light: **#F2EFEA** (RGB: 242, 239, 234)
- Dark: **#162622** (RGB: 22, 38, 34)

Surface:

- Light: **#FFFFFF** (RGB: 255, 255, 255)
- Dark: **#1E2E2A** (RGB: 30, 46, 42)

Text Primary:

- Light: **#1F2933** (RGB: 31, 41, 51)
- Dark: **#E6ECEA** (RGB: 230, 236, 234)

Text Muted:

- Light: **#6B7280** (RGB: 107, 114, 128)
- Dark: **#9AA5A1** (RGB: 154, 165, 161)

Border:

- Light: **#E2E2E2** (RGB: 226, 226, 226)
- Dark: **#2C3F3A** (RGB: 44, 63, 58)

4.4 Add Status Colors

Create color sets in **Status/** folder:

available:

- Light: **#1F4D3A**, Dark: **#4FA58A**

purchased:

- Light: **#2F7F6D**, Dark: **#63BFA8**

partial:

- Light: **#D6B25E**, Dark: **#E4C76A**

shipped:

- Light: **#3A7CA5**, Dark: **#6FAED6**

wrapped:

- Light: **#B23A48**, Dark: **#E06A75**

gifted:

- Light: **#C9A227**, Dark: **#F0D97C**
-

Step 5: Create AppConfigurration

5.1 Create AppConfigurration.swift

In **Core/Configuration/**:

```
// Core/Configuration/AppConfiguration.swift

import Foundation

/// Global application configuration loaded from xcconfig
struct AppConfigurration {

    // MARK: - API Configuration

    /// API base URL loaded from xcconfig
    /// Production: https://api.christmastracker.com
    /// QA: https://api-qa.christmastracker.com
    /// Debug: https://api-dev.christmastracker.com
    static let apiBaseURL: URL = {
        guard let urlString = Bundle.main.infoDictionary?["API_BASE_URL"] as? String,
              let url = URL(string: urlString) else {
            fatalError("API_BASE_URL not configured in xcconfig")
        }
    }
```

```

    return url
}()

// MARK: - Cache Configuration

/// Duration to cache network responses (seconds)
/// Production: 900 (15 minutes)
/// Debug: 30 (30 seconds)
static let cacheDuration: TimeInterval = {
    guard let value = Bundle.main.infoDictionary?["CACHE_DURATION"] as? String,
        let duration = TimeInterval(value) else {
        return 900 // Default 15 minutes
    }
    return duration
}()

// MARK: - Session Configuration

/// Session timeout duration (seconds)
/// Production: 7200 (2 hours)
/// Debug: 300 (5 minutes)
static let sessionTimeout: TimeInterval = {
    guard let value = Bundle.main.infoDictionary?["SESSION_TIMEOUT"] as? String,
        let duration = TimeInterval(value) else {
        return 7200 // Default 2 hours
    }
    return duration
}()

/// Background refresh threshold (seconds)
/// Production: 900 (15 minutes)
/// Debug: 60 (1 minute)
static let backgroundRefreshThreshold: TimeInterval = {
    guard let value = Bundle.main.infoDictionary?["BACKGROUND_REFRESH"] as? String,
        let duration = TimeInterval(value) else {
        return 900 // Default 15 minutes
    }
    return duration
}()

// MARK: - Logging Configuration

/// Log flush interval (seconds)
/// Production: 60 (1 minute)
/// Debug: 15 (15 seconds)
static let logFlushInterval: TimeInterval = {
    guard let value = Bundle.main.infoDictionary?["LOG_FLUSH_INTERVAL"] as? String,
        let duration = TimeInterval(value) else {
        return 60 // Default 60 seconds
    }
    return duration
}()

// MARK: - Environment

/// Current environment (debug or production)
static let environment: String = {
    #if DEBUG
    return "debug"
    #else
    return "production"
    #endif
}()

```



```
/// Whether running in debug mode
static let isDebug: Bool = {
    #if DEBUG
        return true
    #else
        return false
    #endif
}()
}
```

Step 6: Create Design System Files

6.1 Create AppColors.swift

In `DesignSystem/Colors/`:

```
// DesignSystem/Colors/AppColors.swift

import SwiftUI

extension Color {

    // MARK: - Brand Colors

    /// Primary brand color (Evergreen)
    /// Light: #1F4D3A, Dark: #4FA58A
    static let primaryColor = Color("primary")

    /// Accent color (Cranberry)
    /// Light: #B23A48, Dark: #E06A75
    static let accentColor = Color("accent")

    /// Highlight color (Warm Gold)
    /// Light: #D6B25E, Dark: #E4C76A
    static let highlightColor = Color("highlight")

    // MARK: - Background Colors

    /// Primary background color
    /// Light: #FAF9F7, Dark: #0F1A17
    static let backgroundPrimaryColor = Color("background-primary")

    /// Secondary background color
    /// Light: #F2EFEA, Dark: #162622
    static let backgroundSecondaryColor = Color("background-secondary")

    /// Surface color for cards/modals
    /// Light: #FFFFFF, Dark: #1E2E2A
    static let surfaceColor = Color("surface")

    // MARK: - Text Colors

    /// Primary text color
    /// Light: #1F2933, Dark: #E6ECEA
    static let textPrimaryColor = Color("text-primary")

    /// Muted text color
    /// Light: #6B7280, Dark: #9AA5A1
}
```

```

static let textMutedColor = Color("text-muted")

// MARK: - Border Colors

/// Border and divider color
/// Light: #E2E2E2, Dark: #2C3F3A
static let borderColor = Color("border")

// MARK: - Status Colors

static let statusAvailableColor = Color("available")
static let statusPurchasedColor = Color("purchased")
static let statusPartialColor = Color("partial")
static let statusShippedColor = Color("shipped")
static let statusWrappedColor = Color("wrapped")
static let statusGiftedColor = Color("gifted")
}

```

6.2 Create AppTypography.swift

In `DesignSystem/Typography/`:

```

// DesignSystem/Typography/AppTypography.swift

import SwiftUI

extension Font {

    // MARK: - System Fonts (Inter/SF Pro)

    static let appLargeTitle = Font.system(size: 34, weight: .bold, design: .default)
    static let appTitle1 = Font.system(size: 28, weight: .semibold, design: .default)
    static let appTitle2 = Font.system(size: 22, weight: .semibold, design: .default)
    static let appTitle3 = Font.system(size: 20, weight: .semibold, design: .default)
    static let appHeadline = Font.system(size: 17, weight: .semibold, design: .default)
    static let appBody = Font.system(size: 17, weight: .regular, design: .default)
    static let appCallout = Font.system(size: 16, weight: .regular, design: .default)
    static let appSubheadline = Font.system(size: 15, weight: .regular, design: .default)
    static let appFootnote = Font.system(size: 13, weight: .regular, design: .default)
    static let appCaption1 = Font.system(size: 12, weight: .regular, design: .default)
    static let appCaption2 = Font.system(size: 11, weight: .regular, design: .default)
}

extension Text {

    func appLargeTitle() -> Text {
        self.font(.appLargeTitle)
    }

    func appTitle1() -> Text {
        self.font(.appTitle1)
    }

    func appTitle2() -> Text {
        self.font(.appTitle2)
    }

    func appTitle3() -> Text {
        self.font(.appTitle3)
    }
}

```

```

func appBody() -> Text {
    self.font(.appBody)
}

func appCaption() -> Text {
    self.font(.appCaption1).foregroundColor(.textMutedColor)
}
}

```

6.3 Create AppSpacing.swift

In `DesignSystem/Spacing/`:

```

// DesignSystem/Spacing/AppSpacing.swift

import SwiftUI

enum AppSpacing {
    /// Extra small: 4pt
    static let xs: CGFloat = 4

    /// Small: 8pt
    static let sm: CGFloat = 8

    /// Medium: 12pt
    static let md: CGFloat = 12

    /// Large: 16pt
    static let lg: CGFloat = 16

    /// Extra large: 24pt
    static let xl: CGFloat = 24

    /// 2X large: 32pt
    static let xxl: CGFloat = 32

    /// 3X large: 48pt
    static let xxxl: CGFloat = 48
}

extension EdgeInsets {
    /// Standard card padding (16pt all sides)
    static let appCardPadding = EdgeInsets(
        top: AppSpacing.lg,
        leading: AppSpacing.lg,
        bottom: AppSpacing.lg,
        trailing: AppSpacing.lg
    )

    /// Standard screen padding (24pt vertical, 16pt horizontal)
    static let appScreenPadding = EdgeInsets(
        top: AppSpacing.xl,
        leading: AppSpacing.lg,
        bottom: AppSpacing.xl,
        trailing: AppSpacing.lg
    )
}

```

Step 7: Create Protocol Definitions

7.1 Create HTTPMethod.swift

In `Core/Networking/`:

```
// Core/Networking/HTTPMethod.swift

import Foundation

enum HTTPMethod: String, Sendable {
    case get = "GET"
    case post = "POST"
    case put = "PUT"
    case delete = "DELETE"
    case patch = "PATCH"
}
```

7.2 Create APIClient Protocol

In `Core/Networking/`:

```
// Core/Networking/APIClient.swift

import Foundation

/// Protocol defining API client interface
protocol APIClient: Sendable {
    /// Perform a network request
    /// - Parameters:
    ///   - method: HTTP method
    ///   - path: API endpoint path
    ///   - body: Optional request body
    ///   - forceRefresh: Skip cache if true
    /// - Returns: Decoded response
    func request<T: Decodable>(
        _ method: HTTPMethod,
        path: String,
        body: (any Encodable & Sendable)?,
        forceRefresh: Bool
    ) async throws -> T
}

enum APIError: Error, Sendable {
    case invalidResponse
    case httpError(statusCode: Int, data: Data)
    case decodingError(Error)
}
```

7.3 Create Repository Protocols

In `Core/Networking/`:

```
// Core/Networking/Repository.swift

import Foundation
```

```

/// Base protocol for all repositories
protocol Repository: Sendable {
    // Marker protocol
}

/// Example: List repository protocol
/// (Will be fully implemented in Phase 5)
protocol ListRepository: Repository {
    func fetchLists() async throws -> [GiftList]
}

/// Example: Item repository protocol
/// (Will be fully implemented in Phase 6)
protocol ItemRepository: Repository {
    func fetchItems(listId: String) async throws -> [GiftItem]
}

```

7.4 Create LogDestination Protocol

In `Core/Logging/`:

```

// Core/Logging/LogDestination.swift

import Foundation

/// Protocol for log destinations
protocol LogDestination: Sendable {
    /// Log a single event (for immediate critical/error events)
    func log(_ event: LogEvent) async

    /// Flush buffered events
    func flush(_ events: [LogEvent]) async
}

/// Log event model
struct LogEvent: Sendable, Codable {
    let timestamp: Date
    let level: LogLevel
    let category: LogCategory
    let message: String
    let metadata: [String: String]
    let signpost: SignpostInfo?
    let duration: TimeInterval?
    let memoryUsage: UInt64?
}

enum LogLevel: String, Codable, Sendable {
    case debug
    case info
    case warning
    case error
    case critical
}

enum LogCategory: String, Codable, Sendable {
    case network
    case auth
    case ui
    case cache
    case performance
}

```

```

        case business
    }

    struct SignpostInfo: Sendable, Codable {
        let name: String
        let id: UUID
        let state: SignpostState
    }

    enum SignpostState: String, Codable, Sendable {
        case begin
        case end
        case event
    }
}

```

Step 8: Create Mock Data Factory

8.1 Create MockData.swift

In `Preview Content/PreviewData/`:

```

// Preview Content/PreviewData/MockData.swift

import Foundation

/// Mock data factory for SwiftUI previews and tests
enum MockData {

    // MARK: - Users

    static let user1 = User(
        id: "user-1",
        email: "john@example.com",
        name: "John Doe",
        avatarUrl: nil,
        createdAt: Date()
    )

    static let user2 = User(
        id: "user-2",
        email: "jane@example.com",
        name: "Jane Smith",
        avatarUrl: nil,
        createdAt: Date()
    )

    // MARK: - Gift Lists

    static let list1 = GiftList(
        id: "list-1",
        name: "John's Christmas List",
        description: "Gifts for John",
        ownerId: user1.id,
        ownerName: user1.name,
        members: [
            ListMember(
                id: "member-1",
                userId: user1.id,
                userName: user1.name,

```

```

        role: .owner,
        invitationStatus: .accepted
    ),
    ListMember(
        id: "member-2",
        userId: user2.id,
        userName: user2.name,
        role: .member,
        invitationStatus: .accepted
    )
],
createdAt: Date(),
updatedAt: Date()
)

```

```

static let list2 = GiftList(
    id: "list-2",
    name: "Jane's Wishlist",
    description: "Gifts for Jane",
    ownerId: user2.id,
    ownerName: user2.name,
    members: [
        ListMember(
            id: "member-3",
            userId: user2.id,
            userName: user2.name,
            role: .owner,
            invitationStatus: .accepted
        )
    ],
    createdAt: Date(),
    updatedAt: Date()
)

```

```

static let allLists = [list1, list2]

```

```

// MARK: - Gift Items

```

```

static let item1 = GiftItem(
    id: "item-1",
    listId: list1.id,
    name: "Book: Swift Programming",
    description: "Latest edition",
    url: "https://example.com/book",
    imageUrl: nil,
    price: 29.99,
    quantity: 1,
    isPurchased: false,
    isPending: false,
    isOffList: false,
    purchasedBy: nil,
    purchasedAt: nil,
    createdBy: user1.id,
    createdAt: Date(),
    updatedAt: Date()
)

```

```

static let item2 = GiftItem(
    id: "item-2",
    listId: list1.id,
    name: "Headphones",
    description: "Noise cancelling",
    url: nil,

```

```

        imageUrl: nil,
        price: 199.99,
        quantity: 1,
        isPurchased: true,
        isPending: false,
        isOffList: false,
        purchasedBy: user2.id,
        purchasedAt: Date(),
        createdBy: user1.id,
        createdAt: Date(),
        updatedAt: Date()
    )

    static let allItems = [item1, item2]
}

// MARK: - Placeholder Models (Phase 1 only)

/// Placeholder User model (will be replaced in Phase 3)
struct User: Identifiable, Codable, Sendable, Hashable {
    let id: String
    let email: String
    let name: String
    let avatarUrl: String?
    let createdAt: Date
}

/// Placeholder GiftList model (will be replaced in Phase 5)
struct GiftList: Identifiable, Codable, Sendable, Hashable {
    let id: String
    let name: String
    let description: String?
    let ownerId: String
    let ownerName: String
    let members: [ListMember]
    let createdAt: Date
    let updatedAt: Date
}

/// Placeholder ListMember model (will be replaced in Phase 5)
struct ListMember: Identifiable, Codable, Sendable, Hashable {
    let id: String
    let userId: String
    let userName: String
    let role: MemberRole
    let invitationStatus: InvitationStatus
}

enum MemberRole: String, Codable, Sendable {
    case owner
    case coOwner
    case member
    case viewer
}

enum InvitationStatus: String, Codable, Sendable {
    case pending
    case accepted
    case declined
}

/// Placeholder GiftItem model (will be replaced in Phase 6)
struct GiftItem: Identifiable, Codable, Sendable, Hashable {

```



```

let id: String
let listId: String
let name: String
let description: String?
let url: String?
let imageUrl: String?
let price: Decimal?
let quantity: Int
let isPurchased: Bool
let isPending: Bool
let isOffList: Bool
let purchasedBy: String?
let purchasedAt: Date?
let createdBy: String
let createdAt: Date
let updatedAt: Date
}

```

Step 9: Create First SwiftUI Preview

9.1 Create DashboardView (Shell)

In `Features/Dashboard/Views/`:

```

// Features/Dashboard/Views/DashboardView.swift

import SwiftUI

struct DashboardView: View {
    let lists: [GiftList]

    var body: some View {
        NavigationStack {
            ScrollView {
                VStack(spacing: AppSpacing.lg) {
                    ForEach(lists) { list in
                        ListPreviewCard(list: list)
                    }
                }
                .padding(EdgeInsets.appScreenPadding)
            }
            .background(Color.backgroundPrimaryColor)
            .navigationTitle("Dashboard")
        }
    }
}

struct ListPreviewCard: View {
    let list: GiftList

    var body: some View {
        VStack(alignment: .leading, spacing: AppSpacing.md) {
            Text(list.name)
                .font(.appTitle3)
                .foregroundColor(.textPrimaryColor)

            HStack {
                Label("\(list.members.count) members", systemImage: "person.2")
                    .font(.appCaption1)
            }
        }
    }
}

```

```

        .foregroundColor(.textMutedColor)
    }
}
.frame(maxWidth: .infinity, alignment: .leading)
.padding(EdgeInsets.appCardPadding)
.background(Color.surfaceColor)
.cornerRadius(12)
.shadow(color: Color.black.opacity(0.05), radius: 4, x: 0, y: 2)
}
}

#Preview("Light Mode") {
    DashboardView(lists: MockData.allLists)
}

#Preview("Dark Mode") {
    DashboardView(lists: MockData.allLists)
    .preferredColorScheme(.dark)
}

#Preview("Empty State") {
    DashboardView(lists: [])
}

```

9.2 Update ChristmasTrackerApp.swift

```

// App/ChristmasTrackerApp.swift

import SwiftUI

@main
struct ChristmasTrackerApp: App {
    var body: some Scene {
        WindowGroup {
            DashboardView(lists: MockData.allLists)
        }
    }
}

```

9.3 Test Preview

1. Open `DashboardView.swift`
2. Click **Resume** button in preview canvas (right panel)
3. Verify:
 - Light mode shows with cream background
 - Dark mode preview works
 - Colors match design system
 - Typography is readable
 - Spacing looks correct

Step 10: Create First Unit Test

10.1 Create AppConfiguratiOnTests.swift

In `ChristmasTrackerTests/Core/Configuration/`:

```

//

```

```
// ChristmasTrackerTests/Core/Configuration/AppConfigurationTests.swift

import Testing
@testable import ChristmasTracker

@Suite("AppConfiguration Tests")
struct AppConfigurationTests {

    @Test("Cache duration is configured correctly")
    func testCacheDuration() {
        #expect(AppConfiguration.cacheDuration > 0)

        #if DEBUG
        #expect(AppConfiguration.cacheDuration == 30)
        #else
        #expect(AppConfiguration.cacheDuration == 900)
        #endif
    }

    @Test("Session timeout is configured correctly")
    func testSessionTimeout() {
        #expect(AppConfiguration.sessionTimeout > 0)

        #if DEBUG
        #expect(AppConfiguration.sessionTimeout == 300)
        #else
        #expect(AppConfiguration.sessionTimeout == 7200)
        #endif
    }

    @Test("Environment detection works")
    func testEnvironment() {
        #if DEBUG
        #expect(AppConfiguration.environment == "debug")
        #expect(AppConfiguration.isDebugEnabled == true)
        #else
        #expect(AppConfiguration.environment == "production")
        #expect(AppConfiguration.isDebugEnabled == false)
        #endif
    }
}
```

10.2 Run Tests

1. Press **Cmd+U** to run all tests
2. Verify all tests pass (3/3)
3. Check test results in **Test Navigator** (left panel, diamond icon)

Step 11: Commit Initial Setup

11.1 Initialize Git (if not already)

```
cd /path/to/ChristmasTracker
git init
```

11.2 Create .gitignore

```
# Create .gitignore in project root
cat > .gitignore << 'EOF'
# Xcode
*.xcodeproj/*
!*.xcodeproj/project.pbxproj
!*.xcodeproj/xcsshareddata/
!*.xcworkspace/contents.xcworkspacedata
*.xcuserstate
*.xcuserdatad
DerivedData/
.build/

# Swift Package Manager
.swiftpm/
Package.resolved

# macOS
.DS_Store

# Environment
.env
EOF
```

11.3 Commit

```
git add .
git commit -m "Phase 1: Initial project setup with architecture foundation

- Configure Xcode project for iOS 26 + Swift 6
- Add xcconfig files for prod and debug configurations
- Create complete folder structure
- Implement Design System (colors, typography, spacing)
- Add core protocol definitions (APIClient, Repository, LogDestination)
- Create mock data factory
- Add first SwiftUI preview (DashboardView shell)
- Add first unit tests (AppConfiguration)

All Phase 1 deliverables complete."
```

Step 12: Verification Checklist

Before proceeding to Phase 2, verify:

Build & Run

- ☐ Project builds without errors (**Cmd+B**)
- ☐ App runs on iPhone simulator
- ☐ App runs on iPad simulator
- ☐ DashboardView displays with correct colors
- ☐ Dark mode works correctly

Configuration

- ☐ AppConfiguration reads values from xcconfig
- ☐ Debug values different from production
- ☐ Unit tests pass for AppConfiguration

Design System

- ☐ All color sets created in Asset Catalog
- ☐ Colors work in light and dark mode
- ☐ Typography extensions work
- ☐ Spacing constants defined

Protocols

- ☐ HTTPMethod enum defined
- ☐ APIClient protocol defined
- ☐ Repository protocols defined
- ☐ LogDestination protocol defined

Mock Data

- ☐ MockData provides sample users
- ☐ MockData provides sample lists
- ☐ MockData provides sample items
- ☐ Previews work with mock data

Tests

- ☐ Unit tests pass (**Cmd+U**)
- ☐ Test target builds successfully
- ☐ Tests use proper Swift Testing syntax

Git

- ☐ Initial commit completed
- ☐ .gitignore configured
- ☐ No build artifacts committed

Troubleshooting

Issue: Colors not appearing

Solution: Ensure color sets named exactly as in code (e.g., `primary`, not `Primary`)

Issue: xcconfig not applying

Solution:

1. Check **Project Settings → Info → Configurations**
2. Ensure xcconfig file selected for each configuration
3. Clean build folder (**Cmd+Shift+K**)
4. Rebuild (**Cmd+B**)

Issue: Preview not working

Solution:

1. Restart preview canvas (Editor → Canvas)
2. Clean DerivedData: `rm -rf ~/Library/Developer/Xcode/DerivedData/*`
3. Restart Xcode

Issue: Tests not running

Solution:

1. Verify test target selected in scheme
 2. Check **Product → Scheme → Edit Scheme → Test**
 3. Ensure ChristmasTrackerTests target enabled
-

Next Steps

✅ Phase 1 Complete!

You now have a solid foundation with:

- Xcode project configured
- xcconfig setup
- Design system in place
- Core protocols defined
- Mock data for development
- First preview and tests working

Proceed to Phase 2: Network & Security Infrastructure

Phase 2 will build on this foundation to add:

- Actor-based APIClient
 - NetworkCache
 - KeychainService
 - DeviceFingerprintService
 - Logger with destinations
 - Repository implementations
-

Time Tracking

Estimated: 6-8 hours

Actual: ____ hours

Breakdown:

- Xcode setup: ~1 hour
 - xcconfig: ~1 hour
 - Asset Catalog: ~2 hours
 - Code setup: ~2 hours
 - Testing: ~1 hour
 - Documentation: ~1 hour
-

Resources

- [Xcode 26 Release Notes](#)
 - [Swift 6 Concurrency](#)
 - [Asset Catalog Documentation](#)
 - [Swift Testing Framework](#)
-

End of Phase 1 Implementation Guide